

Funktion - isolate_finger(...)

Anforderungen:

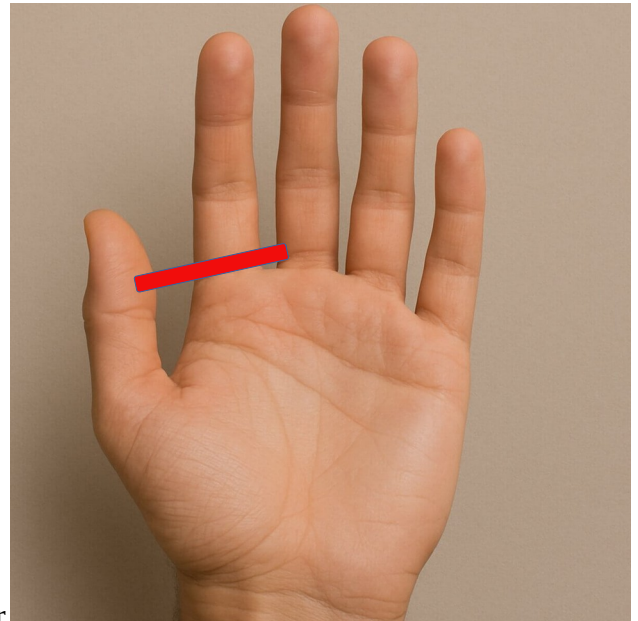
- Abschneiden des verletzten Fingers vom 3D-Scan
- Möglichst wenig Userinput, da dies zu einer hohen Fehlerquote führt
- Schnitt auf Höhe der Fingerwurzel
- Schnitt soll bei allen Untersuchungen am möglichst gleichen Ort passieren, also deterministisch
- Danach deterministisches Ausrichten des Finger, möglichst senkrecht

Grundgedanken:

Am Besten wäre natürlich, den Schnittprozess komplett ohne Userinput entwickeln zu können. Das scheitert aber an zwei Problemen:

1. Wahl des korrekten Fingers
2. Haltung der Finger und Qualität des Scans

Falls aber in Zukunft mit der neuen 3D-Kamera bzw. dem „Kameratunnel“ bessere Scans inklusive besserer Markierungen produziert werden können, kann Problem 1 gelöst werden, indem der Algorithmus den dann einzigen Finger, welcher einige Faces (threshold der Anzahl muss getestet werden) in der Farbe der Markierung auswählt. Problem 2 kann Folgend gelöst werden: Der Patientin oder



dem Patient soll ermutigt werden, die Hand beim Aufnehmen des Scans Zuerst Ausrichtung der Hand, dass sie „aufrecht“ steht. Das kann gelöst werden, indem der Algorithmus die Hand auf die xz-Ebene projiziert. Dann wird die Hand um jeweils 1° gedreht, und die Stellung, welche die kleinste Fläche erzeugt, wird ausgewählt. Die Hand steht nun aufrecht (Oder genau „falsch“ herum, das soll dann mit Fehlerbehandlung gefixt werden). Dann schneidet der Algorithmus die Hand in viele kleine Flächen. Die Schnittebenen sind dann mit der y-Achse als Normalenvektor definiert. Die Schnittbilder ähneln dann auf Höhe der 4 Finger (also vor dem Mittelhand) 4 Kreisen oder Ellipsen. Falls dann die Ellipse des verletzten Fingers schließlich mit der des Nachbarfingers „zusammenwächst“ kann diese Eben als Höhe h festgelegt werden. Daraufhin folgt der Algorithmus dem der etwas weniger qualitativen Scans, sieh dazu unten (1).

Algorithmus – für aktuelle Scans(aktuell nur für Zeige- bis kleiner Finger):

- Lade den Scan mit dem package pyvista
- 3D Scan wird ebenfalls nach oben beschrieben prinzipiell senkrecht platziert
- User wird gebeten, den verletzten Finger auszuwählen in einem Dropdownmenü
- User markiert den Fingernagel des verletzten Fingers
 - Präzision ist dabei unwichtig, da folgender user-error nahezu nicht in das Endergebnis mit einfließt
- User wird aufgefordert, den Fingernagel des benachbarten Fingers auszuwählen, zur Konsistenz:

Verletzter Finger	Wähle aus
Zeigefinger	Mittelfinger
Mittelfinger	Ringfinger
Ringfinger	kl. Finger
kl. Finger	Ringfinger

- Danach wird mit Dijkstras Algorithmus der kürzeste Weg der beiden Punkte berechnet
- (1) Die Höhe des niedrigsten (nach y) Vertice wird als Höhe h gespeichert
- Normale der finalen Schnittfläche wird berechnet:
 - Der vom User ausgewählte Punkt „wächst“ nach dem Prinzip der Breitensuche um eine gesetzte Anzahl an Vertices (Anzahl muss wieder ertestet werden)
 - Berechnen einer Achse s der Punkte mit Hilfe von Principal Component Analysis
 - s folgt jetzt nahezu exakt als „Achse des Fingers“
 - s wird als Normale der Ebene definiert
- Alle Daten sind gesammelt. Die Höhe h des Schnitts ist jetzt möglichst deterministisch berechnet. Dieser Punkt bestimmt auch als „Startpunkt“ des Normalenvektors s , welcher eine Ebene erzeugt und den Finger schneidet.
- Schneiden, neuen 3D Scan laden

Aufruf und Rückgabe:

Aufruf:

isolate_finger(path: str) -> None

Rückgabe:

Keine. Der Algorithmus soll im gegebenen Pfad in dessen Parent, falls nicht vorhanden, ein neues Directory namens „Markierungen“ erstellen und darin ein weiteres Directory mit dem Namen des Parent-Ordners der obj_Datei + „_marked“ erstellen. Falls dies bereits vorhanden soll mit Userinput erfragt werden, ob dieses gelöscht werden darf.

Aktueller Programmierungsstatus:

Arbeite aktuell daran.